MMDU MULLANA OFFICIAL SYLLABUS • BCSE-007 • UNIT 1

Unit 1: Foundations, Array Addressing, Searching & Sorting Algorithms

Subject: Data Structures & Algorithms (BCSE-007) • **Exam Weightage:** 25 - 30 Marks • **Study Time:** 40 mins

Asymptotic notation (O, Ω, Θ), order of growth, Master Theorem, 1D, 2D (Row/Column major) and 3D array memory mapping, sparse matrix triplet representation, and full sorting/searching suite (Linear, Binary, Bubble, Selection, Insertion, Merge, Quick, Heap, Radix).

1. Asymptotic Complexity, Order of Growth & Master Theorem

1. Mathematical Notation & Asymptotic Bounds

In algorithmic analysis, **asymptotic notation** describes the limiting behavior of an algorithm's execution time or memory footprint as input size n approaches infinity:

- **Big-O (O) — Asymptotic Upper Bound:**

$f(n) = O(g(n))$ if and only if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
Represents the mathematical **worst-case** performance guarantee.

- **Big-Omega (Ω) — Asymptotic Lower Bound:**

$f(n) = \Omega(g(n))$ if and only if there exist positive constants $c > 0$ and $n_0 \geq 1$ such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.
Represents the mathematical **best-case** performance bound.

- **Big-Theta (Θ) — Asymptotically Tight Bound:**

$f(n) = \Theta(g(n))$ if and only if there exist positive constants $c_1, c_2 > 0$ and $n_0 \geq 1$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.
Holds when an algorithm is bounded both from above and below by the same growth rate.

2. Standard Hierarchy of Algorithmic Growth Rates

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

3. Master Theorem for Divide-and-Conquer Recurrences

For recurrences of the standard form T(n) = a·T(n/b) + f(n) where a ≥ 1, b > 1, and f(n) = Θ(n^k · log^p n):

1. **Case 1:** If log_b(a) > k ⟹ T(n) = Θ(n^(log_b a)).

2. **Case 2:** If log_b(a) = k:

- If $p > -1 \implies T(n) = \Theta(n^k \cdot \log^{(p+1)} n)$.
- If $p = -1 \implies T(n) = \Theta(n^k \cdot \log \log n)$.

3. **Case 3:** If $\log_b(a) < k$ and regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds $\implies T(n) = \Theta(f(n))$.

2. Multi-Dimensional Array Addressing & Sparse Matrices

1. Sequential Storage & Memory Mapping Formulas

Because physical computer RAM is linear (1D byte-addressable memory), multi-dimensional arrays must be flattened deterministically.

Let:

- B = Base Address (memory address of first element A[L] or $A[L_1][L_2]$).
- W = Width of each element in bytes (e.g., 4 bytes for int, 8 for double).
- L_1, L_2, L_3 = Lower bounds; U_1, U_2, U_3 = Upper bounds.
- $M = (U_1 - L_1 + 1)$ (total rows); $N = (U_2 - L_2 + 1)$ (total columns); $P = (U_3 - L_3 + 1)$ (depth/planes).

1D Array Addressing:

$$Loc(A[i]) = B + (i - L) \times W$$

2D Array Row-Major Order (RMO) — Stored Row by Row:

$$Loc(A[i][j]) = B + [(i - L_1) \times N + (j - L_2)] \times W$$

2D Array Column-Major Order (CMO) — Stored Column by Column:

$$Loc(A[i][j]) = B + [(j - L_2) \times M + (i - L_1)] \times W$$

3D Array Row-Major Order:

$$Loc(A[i][j][k]) = B + [(i - L_1) \times N \times P + (j - L_2) \times P + (k - L_3)] \times W$$

2. Sparse Matrices & Triplet Representation

A **Sparse Matrix** is a matrix where the vast majority (typically ≥ 70%) of elements are zero. Storing all M × N elements wastes massive RAM.

- **Triplet Array Representation (Coordinate List / COO):**

- Stored as a 2D array of size (K + 1) × 3, where K is the number of non-zero elements.
- **Row 0 (Metadata):** [Total_Rows, Total_Cols, Total_NonZeros]
- **Rows 1 to K:** [Row_Index, Col_Index, NonZero_Value]
- Space complexity drops from O(M × N) down to O(K).

3. Searching & Sorting Algorithms Complexity Suite

1. Searching Algorithms Comparison

- **Linear Search:** Scans sequentially from index 0 to n-1. Best: O(1), Worst: O(n), Space: O(1). Works on unsorted collections.

- **Binary Search:** Divide-and-conquer on **sorted** arrays. Computes mid = low + (high - low)/2. Best: O(1), Average/Worst: O(log n), Space: O(1) iterative, O(log n) recursive.

2. Comprehensive Sorting Algorithm Master Matrix

Algorithm	Best Time	Average Time	Worst Time	Space	Stable?	Method / In-Place
Bubble Sort	O(n) [flagged]	$O(n^2)$	$O(n^2)$	O(1)	Yes	Adjacent swaps; in-place
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Min selection; in-place
Insertion Sort	O(n) [sorted]	$O(n^2)$	$O(n^2)$	O(1)	Yes	Shift & insert; in-place
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Divide & Conquer; out-of-place
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$ [skewed]	O(log n)	No	Partitioning; in-place
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Complete binary heap; in-place

| **Radix Sort** | O(d(n + k)) | O(d(n + k)) | O(d(n + k)) | O(n + k) | Yes | Non-comparative digit buckets |